

PITTRACK: SISTEMA DISTRIBUÍDO PARA OFICINAS MECÂNICAS

2018001834 - Gabriel Belluco Perez
2022008537 - Pedro da Silva Furlaneto
2021002000 - Melissa Arantes B. da Silva

SEMINÁRIO

Prof. Rafael Frinhani





PitTrack: Sistema Distribuído para Oficinas Mecânicas

Resumo

O PitTrack é um protótipo distribuído desenvolvido para apoiar o acompanhamento de ordens de serviço em oficinas mecânicas, oferecendo maior transparência durante o processo de manutenção de veículos. A solução busca reduzir a dependência de ligações telefônicas, mensagens instantâneas e registros descentralizados, permitindo que clientes acompanhem em tempo real as etapas da manutenção, aproveitem orçamentos, visualizem mídias e recebam atualizações do andamento do serviço.

A arquitetura foi desenvolvida utilizando React e Vite no frontend, Node.js e Express no backend, PostgreSQL para persistência dos dados, Redis Streams para comunicação assíncrona baseada em eventos, Redis Pub/Sub para distribuição de notificações em tempo real, Socket.IO para atualização da interface e WebRTC para realização de transmissões ao vivo entre oficina e cliente. Além disso, processos independentes (*workers*) são responsáveis pelo processamento assíncrono de eventos relacionados ao diagnóstico, reparo, peças e notificações.

A validação do protótipo foi realizada por meio da execução dos fluxos de negócio, observando o processamento dos eventos distribuídos, a comunicação entre os componentes, a atualização das interfaces em tempo real e o funcionamento dos mecanismos de persistência e notificação. Os resultados demonstraram que a arquitetura proposta atende aos objetivos do projeto, evidenciando os principais conceitos de Sistemas Distribuídos, como desacoplamento, comunicação assíncrona, processamento por eventos e cooperação entre processos independentes.

Palavras-chave: Sistemas Distribuídos. Redis Streams. Comunicação Baseada em Eventos. Socket.IO. WebRTC.

1 Introdução

O acompanhamento da manutenção de veículos ainda é realizado, em muitas oficinas mecânicas, por meio de ligações telefônicas, aplicativos de mensagens e registros descentralizados. Esse modelo dificulta o acompanhamento das etapas do serviço pelo cliente, reduz a transparência do processo, aumenta o retrabalho da equipe e dificulta o registro do histórico completo da manutenção.

Nesse contexto, torna-se relevante o desenvolvimento de soluções que centralizem as informações da ordem de serviço e permitam o acompanhamento das atividades em tempo real. Além de melhorar a comunicação entre oficina e cliente, esse tipo de sistema possibilita registrar evidências da manutenção, acompanhar alterações de status, compartilhar mídias e reduzir falhas decorrentes da comunicação manual.

O projeto PitTrack foi desenvolvido com o objetivo de

construir um protótipo distribuído para gerenciamento de ordens de serviço em oficinas mecânicas. O sistema permite o cadastro de clientes, veículos e ordens de serviço, acompanhamento das etapas da manutenção, geração e aprovação de orçamentos, gerenciamento de peças, envio de fotos e vídeos da manutenção, realização de transmissões ao vivo entre oficina e cliente e atualização das informações em tempo real.

A solução foi desenvolvida utilizando uma arquitetura baseada em sistemas distribuídos, na qual diferentes processos executam responsabilidades específicas e comunicam-se por meio de eventos. Essa abordagem permite desacoplamento entre componentes, comunicação assíncrona, rastreabilidade das alterações realizadas durante a manutenção e atualização em tempo real das interfaces do sistema, características amplamente empregadas em arquiteturas distribuídas [Tanenbaum & van Steen \(2023\)](#).

A arquitetura funcional do PitTrack é composta por um frontend desenvolvido em React, um backend implementado em Node.js e Express, PostgreSQL para persistência das informações, Redis Streams como barramento persistente de eventos, Redis Pub/Sub para distribuição de notificações em tempo real e quatro *workers* independentes responsáveis pelo processamento assíncrono de diagnósticos, peças, reparos e notificações. Além disso, o sistema utiliza Socket.IO para atualização automática das interfaces e WebRTC para realização de transmissões ao vivo entre oficina e cliente.

O objetivo geral deste trabalho é apresentar o desenvolvimento do PitTrack e demonstrar a aplicação dos principais conceitos de Sistemas Distribuídos na construção de uma solução para acompanhamento de ordens de serviço em oficinas mecânicas.

Este relatório está organizado da seguinte forma. A Seção 2 apresenta a fundamentação teórica utilizada no projeto. A Seção 3 descreve a metodologia adotada, a modelagem da arquitetura e a implementação do sistema. A Seção 4 apresenta a avaliação do protótipo desenvolvido. Por fim, a Seção 5 apresenta as conclusões obtidas e as possibilidades de evolução do sistema.

2 Referencial Teórico

Os sistemas distribuídos são caracterizados pela cooperação entre múltiplos processos independentes que se comunicam por meio de uma rede para oferecer uma funcionalidade única ao usuário. Essa abordagem permite distribuir responsabilidades entre diferentes componentes da aplicação, favorecendo escalabilidade, desacoplamento, disponibilidade e facilidade de manutenção [Tanenbaum & van Steen \(2023\)](#).

No PitTrack, a comunicação entre o frontend e o backend ocorre de forma síncrona por meio de requisições HTTP REST.

Esse modelo é utilizado para operações iniciadas pelo usuário, como criação de ordens de serviço, atualização das etapas da manutenção, geração de orçamentos, cadastro de peças e envio de mídias.

Para a comunicação entre os processos internos da aplicação foi adotado um modelo baseado em eventos utilizando Redis Streams. Sempre que ocorre uma alteração relevante no estado do sistema, a API publica um evento no barramento de eventos. Os diferentes *workers* monitoram continuamente esse fluxo e executam apenas as atividades relacionadas à sua responsabilidade, promovendo desacoplamento entre os componentes da aplicação [Redis Ltd. \(2026\)](#).

O Redis Streams foi escolhido por oferecer persistência dos eventos publicados, permitindo que mensagens permaneçam armazenadas até serem processadas pelos consumidores. Essa característica possibilita rastreabilidade das alterações realizadas durante o fluxo da manutenção e reduz o risco de perda de eventos em aplicações distribuídas [Redis Ltd. \(2026\)](#).

Além do Redis Streams, o projeto utiliza Redis Pub/Sub para distribuição de notificações em tempo real. Enquanto o Streams é responsável pelo armazenamento persistente dos eventos de negócio, o Pub/Sub é utilizado apenas para entrega imediata das notificações aos clientes conectados. No PitTrack, o *notification-worker* consome os eventos registrados no Redis Streams e publica notificações no canal *live-notifications*. A API permanece inscrita nesse canal e retransmite essas notificações para as interfaces utilizando Socket.IO [Redis Ltd. \(2026\)](#); [Socket.IO \(2026\)](#).

O Socket.IO foi adotado para manter comunicação bidirecional entre servidor e clientes, permitindo que alterações ocorridas durante a execução da manutenção sejam refletidas automaticamente nas interfaces da oficina e do cliente, sem necessidade de atualização manual da página [Socket.IO \(2026\)](#).

Além das notificações em tempo real, o sistema utiliza WebRTC para permitir transmissões ao vivo entre oficina e cliente durante etapas específicas da manutenção. Nesse modelo, o vídeo é transmitido diretamente entre os navegadores, enquanto o backend utiliza Socket.IO apenas para realizar a sinalização da conexão, trocando mensagens como *offer*, *answer* e *ICE candidates*. Dessa forma, o servidor não participa da transmissão da mídia, apenas coordena o estabelecimento da conexão.

Para armazenamento permanente das informações foi utilizado PostgreSQL. O banco de dados mantém os registros de clientes, veículos, ordens de serviço, histórico de status, orçamentos, peças, substituições, registros de mídia e sessões de transmissão ao vivo. Dessa forma, o PostgreSQL representa o estado persistente da aplicação, enquanto o Redis Streams registra a sequência cronológica dos eventos distribuídos [PostgreSQL Global Development Group \(2026\)](#).

O envio de fotos e vídeos foi implementado utilizando o middleware Multer, responsável pelo recebimento dos arquivos enviados via HTTP. As mídias são armazenadas localmente no servidor, enquanto seus metadados permanecem registrados no PostgreSQL e vinculados à etapa correspondente da ordem de serviço.

Toda a infraestrutura do sistema foi organizada utilizando Docker Compose, permitindo inicializar frontend, backend, PostgreSQL, Redis e os *workers* por meio de uma única configuração, garantindo reprodutibilidade do ambiente de desenvolvimento e simplificando a execução do protótipo

[Docker Inc. \(2026\)](#).

O backend foi desenvolvido utilizando Node.js e Express, enquanto o frontend foi implementado utilizando React e Vite. A escolha dessas tecnologias proporcionou integração simples entre os componentes da aplicação, desenvolvimento rápido da interface e ampla disponibilidade de bibliotecas para implementação dos mecanismos de comunicação distribuída [OpenJS Foundation \(2026b,a\)](#); [Meta Platforms, Inc. \(2026\)](#).

A combinação dessas tecnologias permitiu construir uma arquitetura distribuída baseada em eventos, caracterizada pela separação de responsabilidades entre processos independentes, comunicação assíncrona, atualização em tempo real das interfaces, persistência dos dados e cooperação entre múltiplos componentes da aplicação.

3 Desenvolvimento

Esta seção descreve o processo adotado para analisar o problema, modelar a arquitetura distribuída e implementar o protótipo do sistema PitTrack.

3.1 Análise do Problema

O desenvolvimento do PitTrack teve como objetivo solucionar a dificuldade de acompanhamento da manutenção de veículos em oficinas mecânicas. Em muitos estabelecimentos, as informações referentes ao andamento dos serviços permanecem descentralizadas, sendo compartilhadas por meio de ligações telefônicas, aplicativos de mensagens ou registros internos, dificultando tanto o trabalho da oficina quanto o acompanhamento pelo cliente.

A partir desse cenário, foram identificados como principais requisitos funcionais o cadastro de clientes e veículos, a criação de ordens de serviço, o acompanhamento das etapas da manutenção, a geração e aprovação de orçamentos, o gerenciamento de peças, o envio de fotos e vídeos, a realização de transmissões ao vivo entre oficina e cliente e a disponibilização de notificações em tempo real.

Além dos requisitos funcionais, o projeto buscou atender requisitos não funcionais importantes para uma aplicação distribuída, como desacoplamento entre componentes, comunicação assíncrona baseada em eventos, persistência dos dados, atualização em tempo real da interface, rastreabilidade das alterações realizadas durante a manutenção e facilidade de implantação utilizando contêineres Docker.

Outro objetivo importante foi aproximar o fluxo do sistema ao funcionamento de uma oficina real. Dessa forma, as etapas da manutenção passaram a ser executadas manualmente pelos usuários da oficina, enquanto os *workers* atuam apenas no processamento assíncrono dos eventos publicados pela aplicação.

3.2 Modelagem do Sistema

A arquitetura funcional implementada é composta por um frontend desenvolvido em React com Vite, um backend implementado em Node.js utilizando Express, PostgreSQL para persistência dos dados, Redis Streams para comunicação assíncrona baseada em eventos, Redis Pub/Sub para distribuição de notificações em tempo real e quatro *workers* independentes responsáveis pelo processamento dos eventos distribuídos.

O frontend disponibiliza duas interfaces distintas. A primeira, acessada pela rota */oficina*, é utilizada pelos funcionários da oficina para criar ordens de serviço, registrar diagnósticos, gerar orçamentos, cadastrar peças, enviar mídias e iniciar transmissões ao vivo. A segunda, acessada pela rota */cliente/:id*, permite ao cliente acompanhar o andamento da manutenção, aprovar orçamentos, visualizar a linha do tempo da ordem, acessar as mídias enviadas pela oficina e participar das transmissões ao vivo.

A API centraliza todas as operações da aplicação. Ela recebe as requisições HTTP provenientes do frontend, valida as regras de negócio, persiste os dados no PostgreSQL, registra eventos no Redis Streams e permanece inscrita no Redis Pub/Sub para encaminhar notificações em tempo real às interfaces utilizando Socket.IO.

O PostgreSQL armazena os dados permanentes da aplicação, incluindo clientes, veículos, ordens de serviço, histórico de status, orçamentos, peças, substituições, registros de mídia e informações relacionadas às sessões de transmissão ao vivo.

O Redis Streams atua como barramento persistente de eventos. Sempre que uma alteração relevante ocorre durante o fluxo da manutenção, a API publica um evento correspondente. Os *workers* monitoram continuamente esse fluxo utilizando grupos de consumidores, permitindo processamento assíncrono e desacoplado.

Foram implementados quatro *workers* independentes. O *diagnostic-worker* consome eventos relacionados à criação das ordens de serviço e registra o início do fluxo de diagnóstico. O *repair-worker* reage à aprovação do orçamento pelo cliente, registrando a liberação da etapa de reparo. O *parts-worker* processa os eventos relacionados às peças, simulando sua reserva e atualização de rastreamento. O *notification-worker* consome os eventos registrados no Redis Streams e publica notificações no Redis Pub/Sub.

As notificações publicadas no Redis Pub/Sub são recebidas pela API, que utiliza Socket.IO para distribuí-las automaticamente às interfaces da oficina e do cliente. Além disso, o sistema utiliza WebRTC para estabelecer transmissões ao vivo entre os navegadores, utilizando Socket.IO apenas para realizar a sinalização da conexão.

3.3 Implementação

O sistema foi desenvolvido utilizando JavaScript como linguagem principal, integrando frontend, backend e *workers* em uma arquitetura distribuída baseada em eventos.

O frontend foi implementado utilizando React com Vite e disponibiliza duas interfaces independentes: uma destinada à oficina e outra destinada ao cliente. A comunicação com o backend ocorre por meio de requisições HTTP para operações de negócio e Socket.IO para recebimento de notificações em tempo real.

A API REST foi desenvolvida utilizando Node.js e Express. Ela concentra toda a lógica de integração da aplicação, sendo responsável pela validação das operações, persistência dos dados no PostgreSQL, publicação de eventos no Redis Streams, gerenciamento das notificações e sinalização das sessões WebRTC.

Sempre que uma operação altera o estado da ordem de serviço, a API registra as informações no PostgreSQL e publica um evento correspondente no Redis Streams. Esse mecanismo

permite que os *workers* processem suas responsabilidades sem dependência direta da API.

Os principais eventos utilizados pela aplicação são *SERVICE_ORDER_CREATED*, *STATUS_UPDATED*, *DIAGNOSIS_STARTED*, *DIAGNOSIS_FINISHED*, *BUDGET_CREATED*, *BUDGET_APPROVED*, *PART_RESERVED*, *PART_TRACKING_UPDATED*, *PART_REPLACED*, *MEDIA_UPLOADED*, *VIDEO_UPLOADED*, *LIVE_STARTED*, *LIVE_ENDED*, *REPAIR_STARTED*, *FINAL_TEST_STARTED* e *MAINTENANCE_FINISHED*.

O envio de mídias foi implementado utilizando Multer. Os arquivos enviados pela oficina são armazenados localmente no servidor, enquanto seus metadados permanecem registrados no PostgreSQL e vinculados à etapa correspondente da ordem de serviço. Após o processamento do upload, a API publica o evento *MEDIA_UPLOADED* no Redis Streams.

As transmissões ao vivo utilizam WebRTC para comunicação direta entre os navegadores da oficina e do cliente. O backend participa apenas da sinalização da conexão utilizando Socket.IO, sem realizar o transporte do fluxo de vídeo.

Toda a infraestrutura do sistema foi organizada utilizando Docker Compose, permitindo inicializar frontend, backend, PostgreSQL, Redis e os quatro *workers* por meio de uma única configuração.

A validação da implementação foi realizada observando a execução simultânea dos contêineres responsáveis pelos serviços da aplicação e o funcionamento das interfaces da oficina e do cliente durante a demonstração do sistema.

4 Resultados

A avaliação do PitTrack teve como objetivo verificar se a arquitetura distribuída implementada atende ao fluxo de manutenção proposto e se os diferentes componentes da aplicação comunicam-se corretamente por meio dos mecanismos distribuídos adotados.

Os testes concentraram-se na execução do fluxo completo da aplicação, observando o comportamento da API, dos *workers*, dos mecanismos de comunicação distribuída e da atualização em tempo real das interfaces da oficina e do cliente.

4.1 Validação do Fluxo Distribuído

O primeiro cenário validado correspondeu à criação de uma ordem de serviço. Após o cadastro realizado pela oficina, a API registra as informações no PostgreSQL e publica o evento *SERVICE_ORDER_CREATED* no Redis Streams.

Esse evento é consumido pelo *diagnostic-worker*, que registra o início do fluxo de diagnóstico e evidencia o processamento assíncrono da arquitetura. Diferentemente das primeiras versões do protótipo, o avanço das etapas da manutenção ocorre manualmente pela oficina, aproximando o sistema do funcionamento de um ambiente real.

Quando o orçamento é gerado e aprovado pelo cliente, a API publica o evento *BUDGET_APPROVED*. O *repair-worker* consome esse evento e registra a liberação da etapa de reparo.

Da mesma forma, quando uma peça é cadastrada durante a manutenção, a API publica o evento correspondente no Redis Streams. O *parts-worker* consome esse evento, simula a reserva da peça e atualiza seu rastreamento por meio do evento *PART_TRACKING_UPDATED*.

O *notification-worker* monitora continuamente o Redis Streams e publica notificações no Redis Pub/Sub, permitindo que todas as alterações relevantes sejam encaminhadas às interfaces em tempo real.

Esse fluxo demonstrou o correto funcionamento da comunicação baseada em eventos, evidenciando o desacoplamento entre a API e os processos responsáveis pelo processamento assíncrono da aplicação.

4.2 Validação das Funcionalidades

Durante os testes foi possível validar o funcionamento completo do fluxo de manutenção implementado pelo sistema.

A interface da oficina permitiu cadastrar ordens de serviço, registrar diagnósticos, gerar orçamentos, solicitar peças, enviar mídias e iniciar transmissões ao vivo.

Já a interface do cliente possibilitou acompanhar a evolução da manutenção em tempo real, aprovar orçamentos, visualizar mídias enviadas pela oficina e participar das transmissões ao vivo quando iniciadas.

Também foram validados o envio de fotos e vídeos utilizando Multer, o armazenamento dos metadados no PostgreSQL e a comunicação entre os navegadores utilizando WebRTC para realização das transmissões ao vivo.

As notificações distribuídas utilizando Redis Pub/Sub e Socket.IO permitiram que todas as alterações realizadas durante o fluxo fossem refletidas automaticamente nas interfaces, sem necessidade de atualização manual da página.

4.3 Evidências da Execução

Durante a execução do protótipo foi possível verificar o funcionamento integrado dos componentes distribuídos da aplicação. Os testes confirmaram a correta persistência das informações no PostgreSQL, a publicação e o consumo de eventos pelo Redis Streams, o processamento independente realizado pelos *workers* e o envio de notificações em tempo real utilizando Redis Pub/Sub e Socket.IO.

Também foi validado o fluxo completo de manutenção, incluindo criação da ordem de serviço, diagnóstico, geração e aprovação de orçamento, gerenciamento de peças, reparo, testes finais, envio de mídias e atualização das interfaces da oficina e do cliente.

Esses resultados demonstram que a arquitetura distribuída implementada atende aos objetivos propostos para o projeto e evidencia, na prática, a utilização dos principais conceitos estudados na disciplina de Sistemas Distribuídos.

4.4 Limitações

Embora o protótipo atenda aos objetivos propostos para a disciplina, algumas limitações permanecem.

A aplicação não implementa autenticação completa de usuários, utiliza armazenamento local para as mídias enviadas, emprega simulações para algumas operações relacionadas ao gerenciamento de peças e não possui garantia transacional forte entre PostgreSQL e Redis Streams. Além disso, as transmissões utilizando WebRTC dependem das permissões e restrições impostas pelo navegador.

4.5 Possibilidades de Evolução

Como trabalhos futuros, o sistema poderá incorporar autenticação com diferentes perfis de usuários, armazenamento distribuído das mídias, integração com serviços reais de rastreamento de peças, mecanismos de observabilidade, testes automatizados, implantação em ambientes de computação em nuvem e melhorias relacionadas à tolerância a falhas e escalabilidade da arquitetura.

De forma geral, os testes realizados demonstraram que a arquitetura distribuída implementada atende ao fluxo proposto pelo projeto, evidenciando a utilização de comunicação baseada em eventos, processamento assíncrono por *workers* independentes, atualização em tempo real das interfaces e co-operação entre os diferentes componentes da aplicação.

5 Conclusões

O projeto PitTrack teve como objetivo desenvolver um protótipo de sistema distribuído para acompanhamento de manutenção de veículos em oficinas mecânicas. A solução foi proposta para minimizar problemas comuns desse contexto, como a falta de transparência no andamento dos serviços, a comunicação descentralizada entre oficina e cliente e a dificuldade de acompanhar as diferentes etapas da manutenção.

A arquitetura desenvolvida permitiu demonstrar diversos conceitos estudados na disciplina de Sistemas Distribuídos. A separação das responsabilidades entre frontend, backend, banco de dados, mecanismos de comunicação distribuída e *workers* tornou o sistema mais organizado e evidenciou a utilização conjunta de comunicação síncrona e assíncrona em uma aplicação prática.

A utilização do Redis Streams como mecanismo de registro e distribuição de eventos persistentes e do Redis Pub/Sub para notificações em tempo real possibilitou o desacoplamento entre os componentes e a atualização automática das interfaces por meio do Socket.IO. O PostgreSQL foi utilizado para armazenar o estado permanente da aplicação, enquanto os *workers* executaram de forma independente o processamento assíncrono dos eventos relacionados ao diagnóstico, reparo, gerenciamento de peças e distribuição das notificações. Além disso, a utilização de WebRTC permitiu implementar transmissões ao vivo entre oficina e cliente, enquanto o envio de fotos e vídeos tornou possível registrar evidências reais da manutenção.

Durante o desenvolvimento, os principais desafios estiveram relacionados à definição da arquitetura distribuída, à integração entre os diferentes componentes, ao gerenciamento da comunicação baseada em eventos e à sincronização entre os processos independentes que compõem a solução. Apesar dessas dificuldades, foi possível implementar um protótipo funcional capaz de demonstrar os principais conceitos abordados na disciplina.

Como possibilidades de evolução, o sistema poderá incorporar autenticação de usuários, diferentes níveis de acesso, armazenamento distribuído das mídias, integração com serviços reais de rastreamento de peças, testes automatizados, ferramentas de monitoramento e observabilidade, além da implantação em ambientes de computação em nuvem.

Conclui-se que o PitTrack atingiu o objetivo proposto ao demonstrar, de forma prática, a aplicação dos principais con-

ceitos de Sistemas Distribuídos em um cenário de manutenção veicular, utilizando comunicação baseada em eventos, processamento assíncrono, atualização em tempo real e cooperação entre processos independentes como base para uma solução funcional e passível de evolução.

6 Contribuição dos Integrantes

A divisão das atividades entre os integrantes do grupo ocorreu de forma colaborativa durante o desenvolvimento do projeto.

1. Membro 1 – Coordenador do Projeto (Melissa)

Responsável pela coordenação das atividades da equipe, organização do cronograma, acompanhamento do desenvolvimento do projeto e integração das contribuições dos integrantes. Também foi responsável pela elaboração e organização do relatório técnico-científico, adaptação ao template em $\LaTeX$, revisão do conteúdo e preparação da versão final para entrega.

2. Membro 2 – Pesquisa e Definição Tecnológica (Pedro e Melissa)

Responsáveis pela pesquisa e análise das tecnologias utilizadas no projeto, incluindo React, Vite, Node.js, Express, PostgreSQL, Redis Streams, Redis Pub/Sub, Socket.IO, WebRTC, Multer e Docker Compose. Também participaram da definição da arquitetura baseada em eventos, da escolha das tecnologias empregadas e da fundamentação técnica da solução distribuída implementada.

3. Membro 3 – Arquiteto do Sistema Distribuído (Pedro e Gabriel)

Responsáveis pela modelagem da arquitetura distribuída do PitTrack, definição dos componentes da aplicação, especificação do fluxo de comunicação entre frontend, API, banco de dados e *workers*, além da definição da utilização de Redis Streams para processamento assíncrono dos eventos e Redis Pub/Sub para distribuição das notificações em tempo real.

4. Membro 4 – Desenvolvedor do Protótipo (Pedro e Gabriel)

Responsáveis pela implementação do sistema, incluindo o desenvolvimento da API REST em Node.js e Express, da interface web em React com Vite, da integração com PostgreSQL, da implementação dos *workers* (*diagnostic-worker*, *repair-worker*, *parts-worker* e *notification-worker*), da comunicação via Redis Streams e Redis Pub/Sub, da atualização em tempo real utilizando Socket.IO, da implementação do envio de mídias utilizando Multer, da integração da funcionalidade de transmissões ao vivo utilizando WebRTC e da configuração do ambiente com Docker Compose.

5. Membro 5 – Testes e Avaliação do Sistema (Pedro e Gabriel)

Responsáveis pela validação do funcionamento do sistema distribuído, execução dos cenários de teste, verificação da comunicação entre os componentes, análise dos eventos registrados no Redis Streams, validação das notificações em tempo real, avaliação do fluxo completo da manutenção e preparação das evidências experimentais utilizadas na demonstração do projeto.

Disponibilidade do Código

O código-fonte desenvolvido neste projeto está disponível no repositório:

<https://github.com/GabrielBelluco/pittrack-xrsc09>

Referências

Docker Inc. (2026). Docker documentation. <https://docs.docker.com/>. Acesso em: 1 jul. 2026.

Meta Platforms, Inc. (2026). React documentation. <https://react.dev/>. Acesso em: 1 jul. 2026.

OpenJS Foundation (2026a). Express documentation. <https://expressjs.com/>. Acesso em: 1 jul. 2026.

OpenJS Foundation (2026b). Node.js documentation. <https://nodejs.org/docs/latest/api/>. Acesso em: 1 jul. 2026.

PostgreSQL Global Development Group (2026). PostgreSQL documentation. <https://www.postgresql.org/docs/>. Acesso em: 1 jul. 2026.

Redis Ltd. (2026). Redis documentation. <https://redis.io/docs/>. Acesso em: 1 jul. 2026.

Socket.IO (2026). Socket.io documentation. <https://socket.io/docs/v4/>. Acesso em: 1 jul. 2026.

Tanenbaum, A. S. & van Steen, M. (2023). *Distributed Systems*. Pearson, 4 edição.

